вМинистерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Пермский национальный исследовательский
политехнический университет»**

Электротехнический факультет
Кафедра «Информационные технологии и автоматизированные системы»
направление подготовки: 09.03.01 – «Информатика, вычислительная
техника»

УТВЕРЖДАЮ
Зав. кафедрой ИТАС

докт. экон. наук, профессор

_____(Р.А.Файзрахманов)

«___» _____ 2026 г.

Дискретная математика
Лабораторная работа №3

Выполнил студент гр. ИВТ-25-1б

Тагиров Вадим Эльвирович
(Фамилия, Имя, Отчество)

_____
(подпись)

Проверил:

Ст. Преп. Каф. ИТАС
Рустамханова. Г. И.
(должность, Ф.И.О руководителя от кафедры)

_____ _____
(оценка) (подпись)

_____
(дата)

г. Пермь 2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....3
1 Теоретическая часть.....4
1.1 Основные понятия алгебры логики.....4
1.2 Метод Квайна.....4
2 Практическая часть.....5
2.1 Описание решение.....5
2.2 Алгоритмы минимизации.....5
2.3 Формирование импликантной матрицы.....6
3 Результаты тестирования.....7
ЗАКЛЮЧЕНИЕ.....10
Приложение.....11

ВВЕДЕНИЕ

Настоящая лабораторная работа посвящена разработке программного обеспечения для минимизации логических функций методом Квайна.

Программа принимает на вход вектор значений логической функции, автоматически строит таблицу истинности, составляет совершенную дизъюнктивную нормальную форму (СДНФ), выполняет многошаговое склеивание термов, находит простые импликанты и строит импликантную матрицу для последующего определения минимальной дизъюнктивной нормальной формы (МДНФ).

Работа выполнена на языке программирования Python. Программа реализует все этапы метода Квайна без использования сторонних библиотек для логических вычислений.

1 Теоретическая часть

1.1 Основные понятия алгебры логики

Логическая функция принимает на вход набор бинарных переменных (0 или 1) и возвращает одно значение (0 или 1). Функция может быть задана вектором значений, где каждый бит соответствует результату для определенной комбинации входных переменных. Если функция зависит от n переменных, длина вектора составляет 2n.

СДНФ (Совершенная Дизъюнктивная Нормальная Форма) — это формула, представляющая собой дизъюнкцию (логическое ИЛИ) конституент единицы. Конституента единицы строится для каждой строки таблицы истинности, где функция равна 1. Переменная входит в конституенту без отрицания, если её значение равно 1, и с отрицанием, если 0.

1.2 Метод Квайна

Метод Квайна предназначен для минимизации логических функций. Он основан на попарном сравнении термов СДНФ. Если два терма отличаются ровно в одной переменной, они склеиваются, а различающаяся переменная заменяется на дефис «-».

Склеивание идёт по шагам: на каждом шаге сравниваются все пары термов, полученных на предыдущем шаге, и процесс продолжается, пока есть что склеивать. Термы, которые больше не склеиваются, называются *простыми импликантами*, а их дизъюнкция даёт *сокращённую ДНФ*. Для получения минимальной формы строится *импликантная матрица*: её строки соответствуют простым импликантам, столбцы — исходным термам СДНФ, а знак «+» отмечает покрытие терма импликантой. По матрице выделяется *ядро*, после чего подбирается минимальное покрытие оставшихся термов; ядро вместе с ним образует МДНФ.

2 Практическая часть

2.1 Описание решение

Программа построена на двух классах. Класс BoolFunction — описывает одну логическую функцию.

1) Поля: name (имя функции), vector (строка из 0 и 1), n (длина вектора), vars_count (количество переменных), vars (список имен переменных, например, ['a', 'b', 'c']).

2) Методы: term_to_str (преобразование терма в строку вида ¬a¬c), print_truth_table (вывод таблицы), get_sdnf (построение СДНФ), glue (склеивание двух термов), find_implicants (многошаговое склеивание и поиск простых импликант), is_covered (проверка, покрывает ли импликанта терм СДНФ), print_implicant_matrix (вывод матрицы), get_mdnf (поиск минимальной формы).

Класс App — отвечает за интерфейс.

1) Поля: functions (словарь для хранения функций).

2) Методы: add_function (добавление функции), analyze_function (запуск минимизации), manager (главное меню), clear (очистка экрана).

2.2 Алгоритмы минимизации

1) Построение таблицы истинности: номер строки переводится в двоичную систему с помощью bin(i)[2:].zfill(n), что позволяет получить комбинацию переменных для каждой строки.

2) Построение СДНФ: для каждой единицы в векторе генерируется терм, где переменные с 0 записываются с отрицанием (¬a).

3) **Поиск склеек:** реализован цикл `while`, который на каждом шаге сравнивает все пары термов, полученных на предыдущем шаге. Если два терма отличаются ровно в одной позиции, они склеиваются, а различающийся символ заменяется на дефис (`-`), что означает «не имеет значения». Термы, которые не склеились ни с одним другим термом, добавляются в список

простых импликант. Склеивание продолжается до тех пор, пока на очередном шаге не останется ни одной пары.

4) Поиск МДНФ: сначала по импликантной матрице выделяется ядро - простые импликанты, каждая из которых является единственной, покрывающей хотя бы один терм СДНФ. Затем для оставшихся термов перебираются все наборы оставшихся импликант и выбирается самый короткий набор, который покрывает их полностью. Ядро и выбранный набор образуют МДНФ.

2.3 Формирование импликантной матрицы

Метод print_implicant_matrix выводит таблицу. Для каждой импликанты (строки) и каждого исходного терма (столбца) проверяется, совпадают ли все символы, кроме дефисов. Если да, ставится знак «+». На основе матрицы метод get_mdnf находит ядро: для каждого терма СДНФ определяется, какие простые импликанты его покрывают; если терм покрыт только одной импликантой, она включается в ядро. Затем программа определяет термы, которые ядро не покрыло, и перебирает наборы оставшихся импликант. Из наборов, полностью покрывающих эти термы, выбирается самый короткий. В результате выводится МДНФ — дизъюнкция импликант ядра и выбранного набора.

3 Результаты тестирования

Для проверки работоспособности программы было проведено тестирование на примере логической функции f1(a, b, c) от трех переменных с вектором значений 11101101. Внешний вид главного меню программы представлен на рисунке 1.

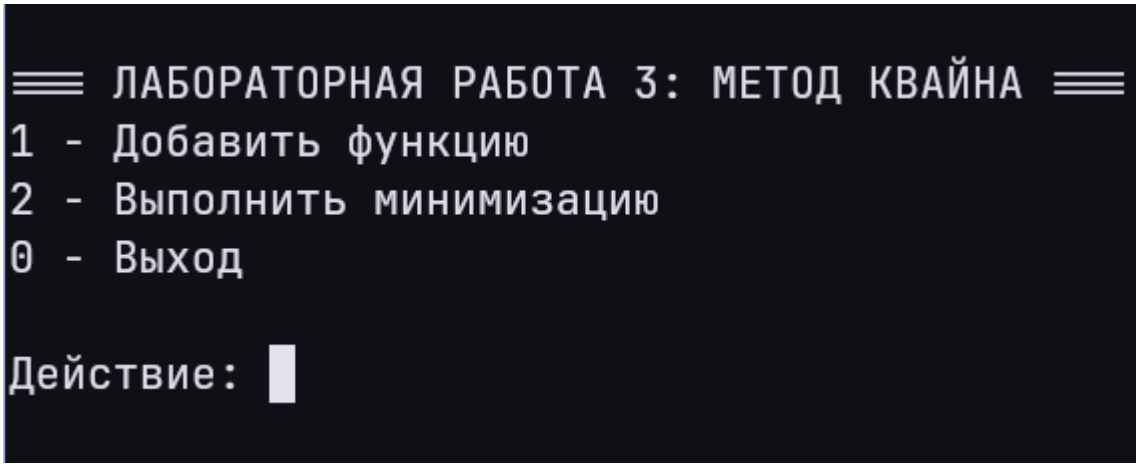


Рисунок 1 — Главное меню программы

Для начала работы необходимо добавить функцию в программу. Процесс ввода имени функции и её вектора значений показан на рисунке 2.

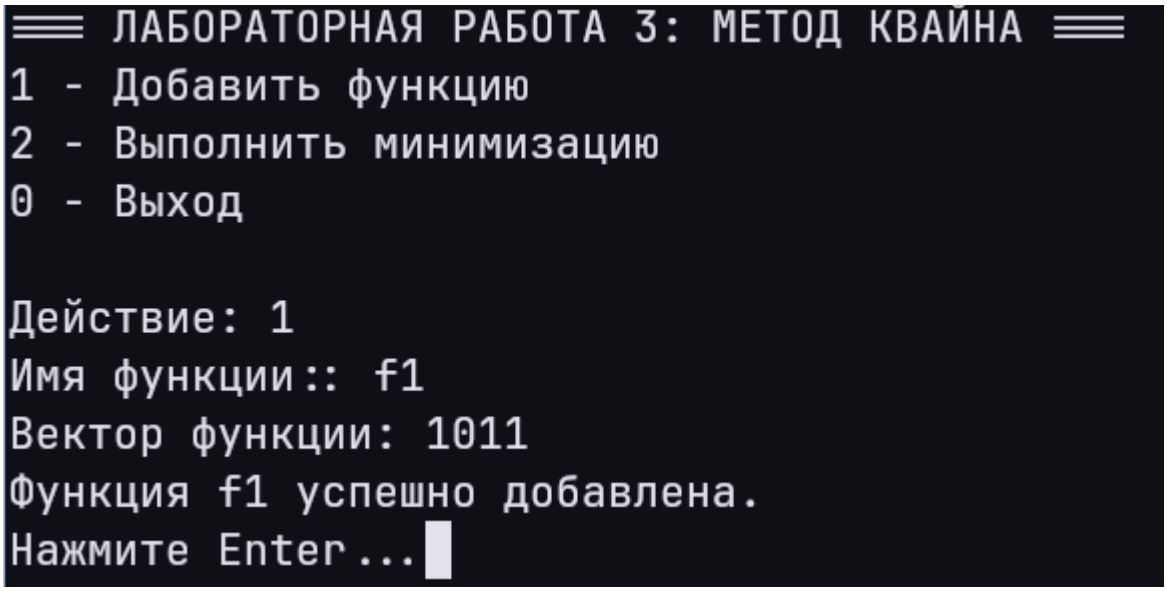


Рисунок 2 — Процесс ввода функции

После ввода программа автоматически строит таблицу истинности и составляет совершенную дизъюнктивную нормальную форму (СДНФ). Вектор 11101101 содержит шесть единиц, поэтому СДНФ состоит из шести

конституент. Результат построения таблицы и СДНФ для функции f1 показан на рисунке 3.

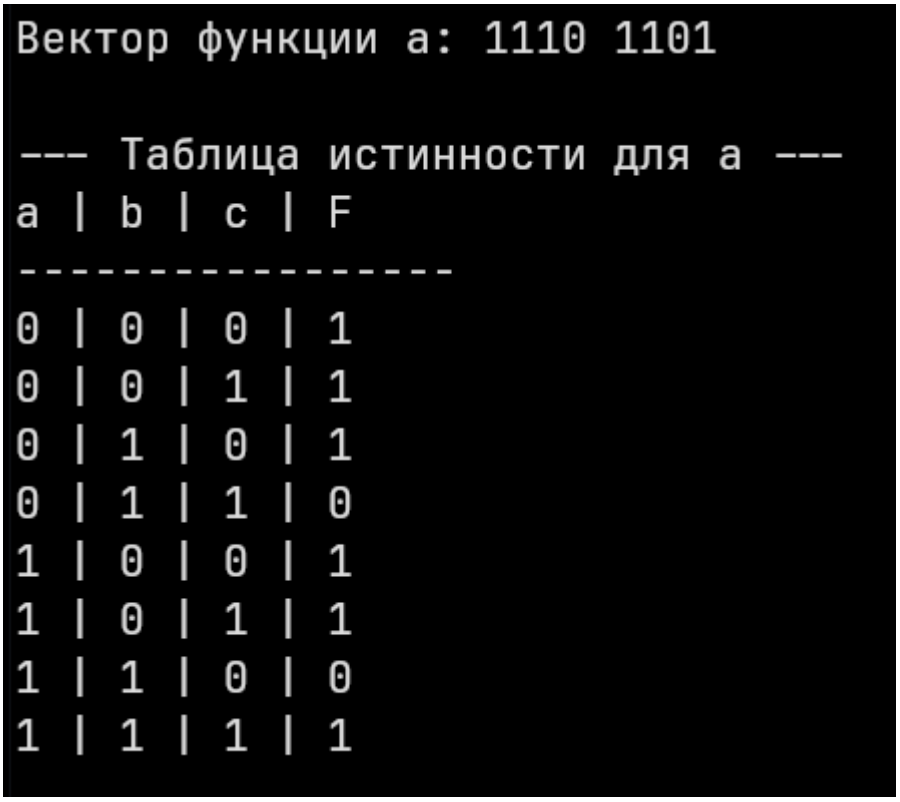


Рисунок 3 — Таблица истинности и СДНФ

Далее программа выполняет многошаговое склеивание термов по методу Квайна. На каждом шаге сравниваются все пары термов, полученных на предыдущем шаге, а термы, которые больше не склеиваются, добавляются в список простых импликант. Для рассматриваемого примера потребовалось два шага склеивания, после чего программа сообщает, что склеивать больше нечего. Полученная сокращённая ДНФ состоит из трёх простых импликант: ¬a¬c ∨ ac ∨ ¬b. Результат склеивания представлен на рисунке 4.

```
--- СДНФ ---
¬a¬b¬c v ¬a¬bc v ¬ab¬c v a¬b¬c v a¬bc v abc

--- Неполные склеивания ---

Шаг 1:
¬a¬b v ¬a¬c v ¬b¬c v ¬bc v a¬b v ac

Шаг 2:
¬b

Шаг 3:
Склеивать больше нечего.

Сокращённая ДНФ:
¬a¬c v ac v ¬b
```

Рисунок 4 — Многошаговое склеивание и сокращённая ДНФ

На основе простых импликант строится импликантная матрица, которая показывает, какие импликанты покрывают исходные конституенты СДНФ. В рассматриваемом примере матрица содержит три строки (простые импликанты ¬a¬c, ac и ¬b) и шесть столбцов (конституенты СДНФ). Внешний вид матрицы показан на рисунке 5.

```
--- Импликантная матрица ---
          | ¬a¬b¬c     | ¬a¬bc      | ¬ab¬c      | a¬b¬c      | a¬bc       | abc        |
-------------------------------------------------------------------------------------------
¬a¬c       | +          |            | +          |            |            |            |
ac         |            |            |            |            | +          | +          |
¬b         | +          | +          |            | +          | +          |            |
```

Рисунок 5 — Импликантная матрица

Завершающим этапом является формирование минимальной дизъюнктивной нормальной формы (МДНФ): программа находит ядро, подбирает минимальное покрытие оставшихся термов и выводит результат.

Для рассматриваемого примера получена МДНФ ¬b ∨ ¬a¬c ∨ ac. Результат вывода МДНФ представлен на рисунке 6.

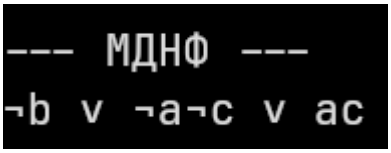


Рисунок 6 — Минимальная ДНФ

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы было разработано программное обеспечение для минимизации логических функций методом Квайна. Программа успешно справляется с поставленной задачей: она автоматически строит таблицу истинности, формирует СДНФ, выполняет многошаговое склеивание и находит простые импликанты, а затем строит импликантную матрицу и определяет минимальную форму.

Разработанный алгоритм реализован без использования готовых библиотек логических вычислений, что позволило глубоко изучить принципы работы метода Квайна. Программа имеет удобный консольный интерфейс, поддерживает ввод функций произвольной размерности (от 1 до 5 переменных) и защищена от некорректного ввода данных. Проведенное тестирование подтвердило корректность работы алгоритмов на различных наборах данных.

Приложение

Полный код лабораторной работы доступен по ссылке:

https://github.com/ZentoTAG/LABS/blob/main/LABS_DM/lab3/main.py